

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and
Commerce
Department of Information Technology (B.Sc.I.T SemesterV)
Applied Business
Practical – X

Roll No.:T010	Name: Sagar Kandari
Class: TYIT	Batch:1
Date of Assignment:26/09/20206	Date/Time of Submission:26/09/2026

Practical 10: Regression Analysis and Time Series Analysis

Part A: Regression Analysis

1. Load the sales_data.csv dataset.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
```

✓ 14.7s
<

```
# 1. Load dataset
df = pd.read_csv("sales_data.csv")
```

✓ 0.0s

2. Display the first five records.

```
# 2. Display first five records
print("First Five Records:")
print(df.head())

# 3 & 4. Independent and dependent variables
X = df[["advertising"]]
y = df["sales"]

print("\nIndependent Variable: Advertising")
print("Dependent Variable: Sales")
```

✓ dot
←

First Five Records:

	advertising	sales
0	10	15
1	12	18
2	15	20
3	18	24
4	20	27

Independent Variable: Advertising
Dependent Variable: Sales

3. Identify the independent and dependent variables.

```
# 3 & 4. Independent and dependent variables
X = df[["advertising"]]
y = df["sales"]
```

Independent Variable: Advertising
Dependent Variable: Sales

4. Select advertising as the independent variable and sales as the dependent variable.

Independent Variable: Advertising
Dependent Variable: Sales

5. Check the data for missing values.

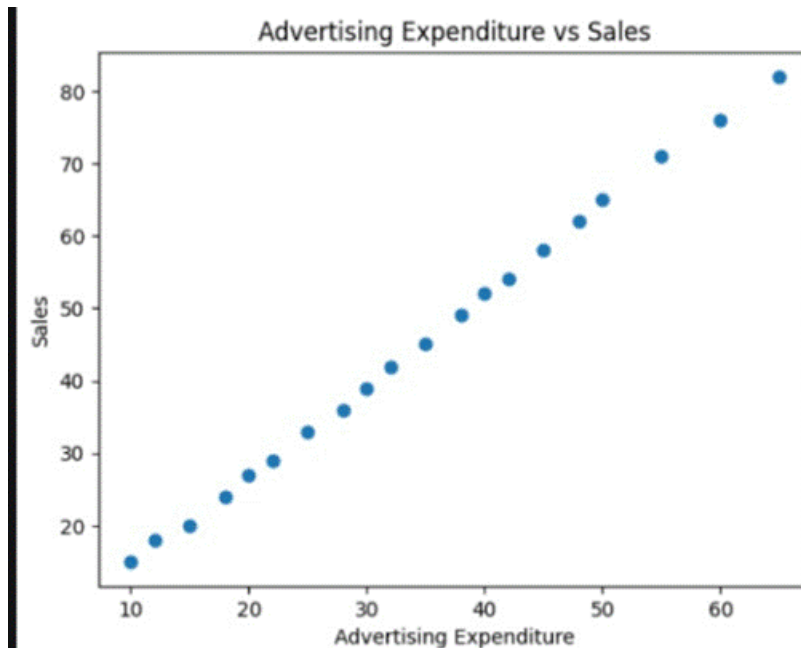
```
# 5. Check missing values
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
advertising    0
sales          0
dtype: int64
```

6. Create a scatter plot of advertising expenditure and sales.

```
# 6. Scatter plot
plt.scatter(df["advertising"], df["sales"])
plt.xlabel("Advertising Expenditure")
plt.ylabel("Sales")
plt.title("Advertising Expenditure vs Sales")
plt.show()
```

✓ 0.2s



7. Develop a simple linear regression model.

```
# 7. Develop Simple Linear Regression Model
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

# 8. Regression equation
b0 = model.intercept_
b1 = model.coef_[0]

print("\nRegression Equation:")
print(f"Sales = {b0:.2f} + {b1:.2f}(Advertising)")
```

✓ 0.4s

```
Regression Equation:  
Sales = 1.75 + 1.24(Advertising)
```

8. Display the regression equation.

```
# 8. Regression equation  
b0 = model.intercept_  
b1 = model.coef_[0]  
  
print("\nRegression Equation:")  
print(f"Sales = {b0:.2f} + {b1:.2f}(Advertising)")  
✓ 0.4s
```

```
Sales = 1.75 + 1.24(Advertising)
```

9. Use the model to predict sales for a given advertising expenditure.

```
# 9. Predict sales for a given advertising expenditure  
advertising_value = 50  
predicted_sales = model.predict([[advertising_value]])  
  
print(f"\nPredicted Sales for Advertising = {advertising_value}: "  
      f"{predicted_sales[0]:.2f}")  
  
# Predictions for test data  
y_pred = model.predict(X_test)
```

```
Predicted Sales for Advertising = 50: 63.96
```

10. Calculate the R^2 score of the model.

```
# 10. R2 Score
r2 = r2_score(y_test, y_pred)
print(f"\nR2 Score: {r2:.4f}")
```

✓ 0.0s

```
R2 Score: 0.9987
c:\Users\Sagar Kandari\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py:2827:
warnings.warn(
```

11. Calculate the Mean Squared Error (MSE).

```
# 11. Mean Squared Error
mse = mean_squared_error(y_test, y_pred)
print(f"MSE: {mse:.4f}")
```

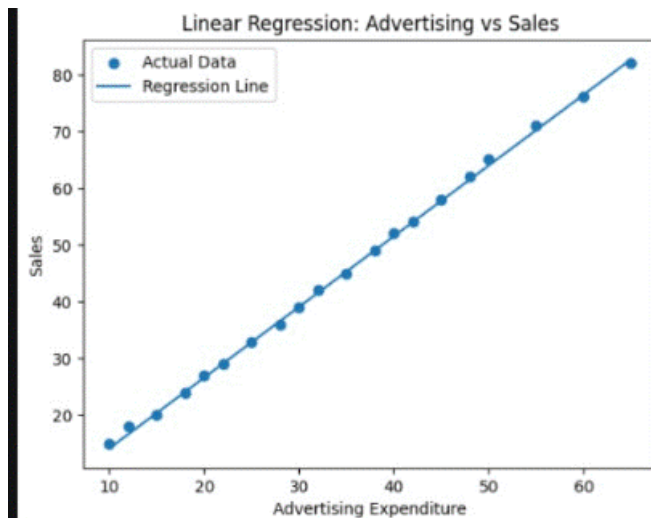
MSE: 0.8345

12. Compare the actual sales with the predicted sales

```
Actual vs Predicted Sales:
  Actual Sales  Predicted Sales
0             15         14.191227
1             71         70.184601
2             62         61.474521
3             18         16.679822
```

13. Plot the regression line with the actual data points.

```
# 13. Plot regression line with actual data
plt.scatter(X, y, label="Actual Data")
plt.plot(X, model.predict(X), label="Regression Line")
plt.xlabel("Advertising Expenditure")
plt.ylabel("Sales")
plt.title("Linear Regression: Advertising vs Sales")
plt.legend()
plt.show()
```



14. Interpret the regression results.

```
# 14. Interpret regression results
print("\nInterpretation:")
print(f"For every 1 unit increase in advertising expenditure, "
      f"sales increase by approximately {b1:.2f} units.")

print(f"The R2 score is {r2:.4f}, which shows how well advertising "
      f"explains the variation in sales.")
```

```
Interpretation:
For every 1 unit increase in advertising expenditure, sales increase by approximately 1.24 units.
The R2 score is 0.9987, which shows how well advertising explains the variation in sales.
```

15. Write a conclusion about how advertising expenditure is related to sales. A simple regression model is:

```
# 15. Conclusion
print("\nConclusion:")
print("Advertising expenditure has a positive relationship with sales.")
print("As advertising expenditure increases, sales generally increase.")
```

Conclusion:

Advertising expenditure has a positive relationship with sales.
As advertising expenditure increases, sales generally increase.

$$\text{Sales} = \beta_0 +$$

$$\beta_1(\text{Advertising Expenditure})$$

:

16. Load monthly sales data.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

✓ 7.7s

17. Convert the date column into date format.

```
# Step 2: Create the monthly sales dataset
data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
              "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"],
    "Sales": [120, 135, 128, 145, 150, 160,
              155, 170, 165, 180, 190, 200]
}

df = pd.DataFrame(data)

print("Monthly Sales Data:")
print(df)
```

```
..
```

Monthly Sales Data:		
	Month	Sales
0	Jan	120
1	Feb	135
2	Mar	128
3	Apr	145
4	May	150
5	Jun	160
6	Jul	155
7	Aug	170
8	Sep	165
9	Oct	180
10	Nov	190
11	Dec	200

18. Plot monthly sales using a line chart.

```
# Step 3: Convert Month column into date format
df["Date"] = pd.to_datetime(df["Month"] + "-2025",
                             format="%b-%Y")

df.set_index("Date", inplace=True)

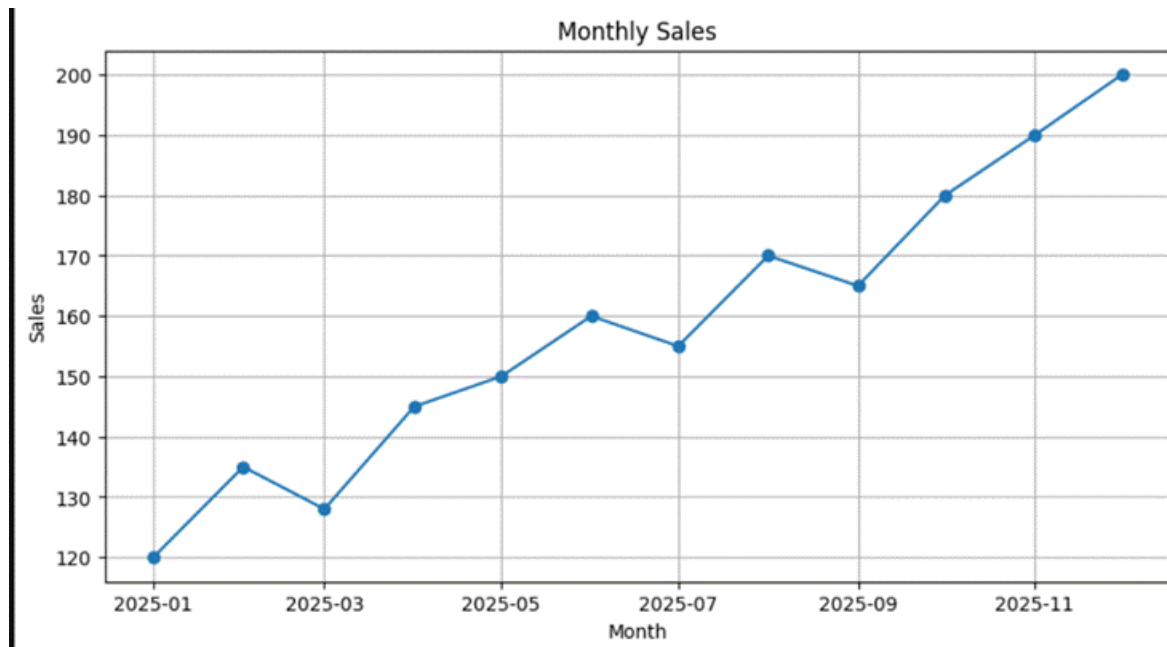
print("\nData after converting to date:")
print(df)
```

✓ 0.0s

Data after converting to date:		
	Month	Sales
Date		
2025-01-01	Jan	120
2025-02-01	Feb	135
2025-03-01	Mar	128
2025-04-01	Apr	145
2025-05-01	May	150
2025-06-01	Jun	160
2025-07-01	Jul	155
2025-08-01	Aug	170
2025-09-01	Sep	165
2025-10-01	Oct	180
2025-11-01	Nov	190
2025-12-01	Dec	200

19. Observe the trend.

```
# Step 4: Plot monthly sales using line chart
plt.figure(figsize=(10, 5))
plt.plot(df.index, df["Sales"], marker="o")
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```



20. Apply Exponential Smoothing.

```
# 5. Apply Exponential Smoothing
es_model = ExponentialSmoothing(
    df["Visitors"],
    trend="add",
    seasonal=None
)

es_fit = es_model.fit()

# 6. Forecast next 3 months
es_forecast = es_fit.forecast(3)
```

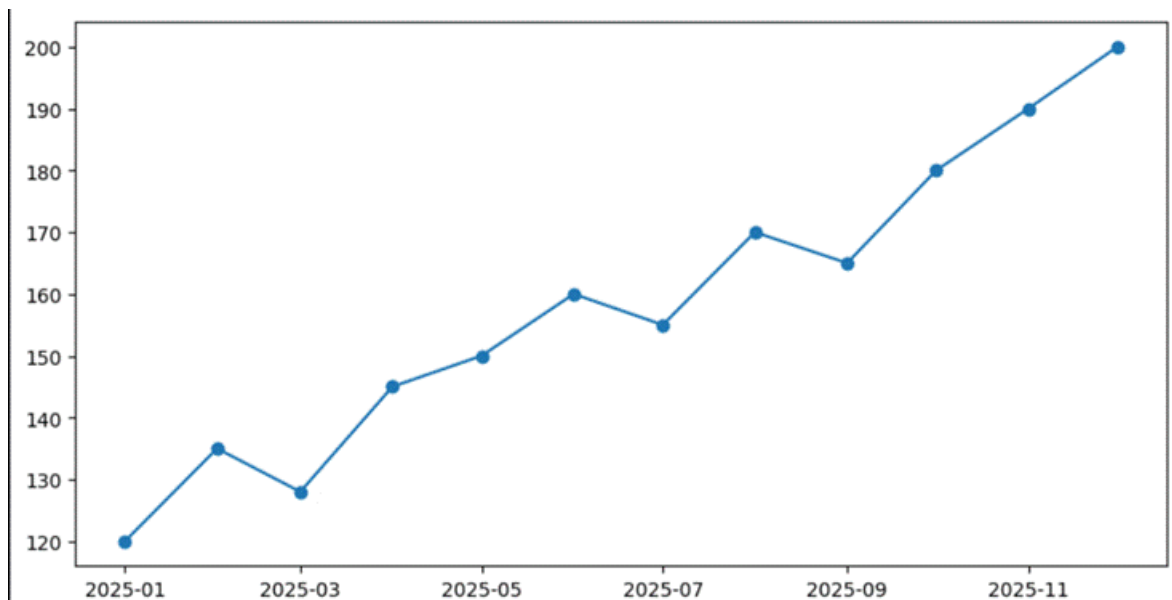
```
Exponential Smoothing Forecast:  
2026-01-01    1603.333352  
2026-02-01    1654.487198  
2026-03-01    1705.641045  
Freq: MS, dtype: float64
```

21. Forecast the next 3 months.

```
# Step 7: Create future dates  
future_dates = pd.date_range(  
    start=df.index[-1] + pd.DateOffset(months=1),  
    periods=3,  
    freq="MS"  
)
```

22. Plot actual and forecasted sales.

```
plt.figure(figsize=(10, 5))  
  
plt.plot(df.index, df["Sales"], marker="o", label="Actual")  
plt.plot(future_dates, forecast_es, marker="o", label="Exponential Smoothing")  
plt.plot(future_dates, forecast_arima, marker="o", label="ARIMA")  
  
plt.title("Sales Forecast")  
plt.xlabel("Month")  
plt.ylabel("Sales")  
  
plt.legend()  
plt.show()
```



23. Calculate forecasting error.

```
# 12. Exponential Smoothing on training data
es_test_model = ExponentialSmoothing(
    train,
    trend="add",
    seasonal=None
)

es_test_fit = es_test_model.fit()

es_test_forecast = es_test_fit.forecast(3)
```

```
Exponential Smoothing:
MAE = 74.67
RMSE = 78.96
```

24. Apply ARIMA to the same data.

```
# 18. Display next 3 months forecast
forecast_table = pd.DataFrame({

    "Month": future_dates.strftime("%b"),

    "Exponential Smoothing":
        np.round(es_forecast.values, 2),

    "ARIMA":
        np.round(arma_forecast.values, 2)
})
```

```
st ----
hing    ARIMA
3.33  1608.71
4.49  1609.66
5.64  1609.76
```

25. Identify which model gives the lower forecasting error for the given dataset. 29. Write the final conclusion.

```
# 19. Final conclusion
if es_mae < arima_mae:

    print("\nFinal Conclusion:")
    print("Exponential Smoothing gives lower forecasting error.")
    print("Therefore, it performs better for this website visitor dataset.")

else:

    print("\nFinal Conclusion:")
    print("ARIMA gives lower forecasting error.")
    print("Therefore, it performs better for this website visitor dataset.")
```

```
ives lower forecasting error.
better for this website visitor dataset.
13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so infer
s, freq)
13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so infer
s, freq)
13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so infer
s, freq)
13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so infer
s, freq)
```